

Experiment no 7

To simulate Network Access Control (NAC) by installing Free RADIUS on Linux, configuring a basic 802.1X authentication setup, and capturing EAP packets using Wireshark.

Objective

Install FreeRADIUS, configure a basic user and client, simulate an 802.1X/EAP authentication request, and analyze the resulting packets using Wireshark.

Prerequisites

- A Linux environment (Ubuntu/Debian recommended).
- sudo (root) privileges.

Phase 1: Installation

Open your terminal. We need to install the RADIUS server, the testing utilities, and Wireshark.

1. Update your package list:

```
sudo apt update
```

2. Install FreeRADIUS, testing tools, and Wireshark:

```
sudo apt install freeradius freeradius-utils wireshark -y
```

*(Note: During Wireshark installation, it may ask if non-superusers should be able to capture packets. Select **Yes**).*

Phase 2: Configuring FreeRADIUS

We need to tell the RADIUS server two things: who is allowed to ask for authentication (the switch/client) and who is allowed to log in (the user).

1. Define the Client (The Authenticator) By default, FreeRADIUS already has your local machine (127.0.0.1) configured as a client with the shared secret testing123. We will use this default setup to keep things simple.

- *File location:* /etc/freeradius/3.0/clients.conf (No changes needed here for this lab).

2. Create a User (The Supplicant) We need to create a test user with a password. Open the users file using a text editor (like nano):

```
sudo nano /etc/freeradius/3.0/users
```

Add the following line at the **very top** of the file:

`student Cleartext-Password := "password123"`

Save and exit (Ctrl+O, Enter, Ctrl+X).

Phase 3: Running FreeRADIUS in Debug Mode

To see exactly what the server is doing, it is best to run it in debug mode rather than in the background.

1. **Stop the background service:**

`sudo systemctl stop freeradius`

2. **Start FreeRADIUS in debug mode:**

`sudo freeradius -X`

Leave this terminal window open. You will see a lot of text scrolling by. Wait until the last line says **Ready to process requests**.

Phase 4: Setting up Wireshark for Capture

Now we will capture the network traffic before we trigger the authentication.

1. Open a **new** terminal window and launch Wireshark (or open it from your applications menu):

`sudo wireshark`

2. In Wireshark, look at the list of interfaces. Double-click on **Loopback: lo** (or 127.0.0.1). This is because our simulation is happening entirely on the local machine.
3. In the display filter bar at the top, type the following to filter out background noise:

`radius || eap`

Press **Enter**. The screen will be blank, waiting for packets.

Phase 5: Simulating 802.1X / EAP Authentication

Go back to your second terminal window (not the one running FreeRADIUS). We will use `radtest` to simulate an EAP (Extensible Authentication Protocol) request.

Run the following command:

`radtest -t eap-md5 student password123 127.0.0.1 0 testing123`

- **student / password123:** The credentials we made in Phase 2.
- **127.0.0.1:** The IP of our RADIUS server.

- **0:** The NAS-Port (simulating a switch port).
- **testing123:** The shared secret between the "switch" and the RADIUS server.
- **-t eap-md5:** Forces the test to use EAP encapsulation, which is what 802.1X uses.

You should see an Access-Accept message in your terminal, meaning the login was successful!

Phase 6: Analyzing the Packets

Go look at your Wireshark window. You should now see several packets listed.

What you are seeing (The EAP Handshake):

1. **Access-Request:** Your radtest command (acting as the switch) sent the user's identity to the RADIUS server. If you expand the RADIUS protocol details in the bottom pane, you will see an EAP-Message attribute containing the user's identity.
2. **Access-Challenge:** The RADIUS server responds, essentially saying, *"I see you, but I need you to prove your identity using EAP-MD5."*
3. **Access-Request:** The client replies with the MD5-hashed password packaged inside an EAP payload.
4. **Access-Accept:** The RADIUS server validates the hash against its database (/etc/freeradius/3.0/users), approves it, and tells the switch to open the network port.

Detailed Message Flow (802.1X/EAP-MD5 Authentication)

1. **Client → FreeRADIUS (UDP Port 1812): RADIUS Access-Request**
 - *Encapsulated EAP Message:* EAP-Response/Identity
 - *Key Data:* User identity ("student"). This packet initiates the authentication process from the RADIUS client (simulating the switch) to the server.
2. **FreeRADIUS → Client (UDP Port 1812): RADIUS Access-Challenge**
 - *Encapsulated EAP Message:* EAP-Request/MD5-Challenge
 - *Key Data:* A random challenge string (nonce) and the requested authentication method (EAP-MD5). The server challenges the client to prove its identity using this specific method.
3. **Client → FreeRADIUS (UDP Port 1812): RADIUS Access-Request**
 - *Encapsulated EAP Message:* EAP-Response/MD5-Challenge
 - *Key Data:* A MD5 hash calculated by combining the user's password ("password123") with the challenge string received in step 2. This proves possession of the correct password without sending it in plaintext.

4. FreeRADIUS → Client (UDP Port 1812): RADIUS Access-Accept

- *Encapsulated EAP Message:* EAP-Success
- *Key Data:* On successful verification of the client's MD5 hash against its own calculation, the server sends this final approval, indicating the user is authenticated.

Conclusion :

This experiment successfully demonstrated 802.1X Network Access Control by simulating a secure EAP-MD5 authentication exchange between a client and a FreeRADIUS server. Furthermore, Wireshark packet analysis validated how client credentials are cryptographically hashed and encapsulated within RADIUS packets to prevent unauthorized network access.